

Learning Path Engine (DAG-based)

1. Feature Name

Learning Path Engine (DAG-based)

2. Purpose

- **Problem Solved:** Courses need structured, ordered content with meaningful progression. A flat list of lessons is not enough — some modules must be completed before others can be attempted.
 - **Why It Exists:** The engine models a learning path as a directed acyclic graph (DAG) of modules. Dependencies between modules define prerequisites, enabling prerequisite-gated content access, meaningful ordering, and clear visual representation of how modules connect.
 - **Role in Application:** It is the core content subsystem. Instructors author paths and modules, students browse/enroll and consume gated content, and the progress/quiz/analytics features all operate on top of these paths and modules.
-

3. What the User Can Do

Guest

- Browse public, published learning paths.

Student

- View a published path and its modules.
- See which modules are completed, unlocked, or locked.
- Open and read any unlocked module's content.
- View the path as an interactive dependency graph.
- Be blocked from modules whose prerequisites are not complete.

Instructor

- Create and edit learning paths.
- Add, edit, delete, publish, unpublish, archive, and reorder modules.
- Add and remove module dependency edges (prerequisites).
- Search and filter modules by title, content type, difficulty, draft, and archived status.

Admin

- All instructor capabilities, plus permanent deletion of paths/modules.
-

4. Feature Workflow

[Instructor creates path]

- POST /api/v1/paths (Admin/Instructor)
- LearningPathService.CreateAsync → path stored (isPublic → also published by default)

[Instructor adds modules + dependencies]

- POST /paths/{id}/modules (add module content, resources, objectives, tags)
- POST /paths/{id}/dependencies (edge module → dependsOnModule; cycle-checked)

[Instructor publishes module]

- PUT /paths/{id}/modules/{moduleId}/publish
- Validation: has title, description, and at least one content item

[Student opens path]

- GET /api/v1/paths/{id}
- Backend computes isUnlocked (all dependencies completed) and isCompleted per module

[Student opens module content]

- GET /paths/{pathId}/modules/{moduleId}
- 403 if prerequisites not completed
- Returns content + previous/next module ids

5. How It Works Internally

Module Unlock Logic (prerequisite gating)

In `LearningPathService.GetByIdAsync()` and `GetModuleContentAsync()`:

1. The path is loaded with modules, their `Dependencies`, and the current user's `Progresses`.
2. `completedModuleIds` = the set of module ids where the user has `Progress.IsCompleted == true`.
3. For each module, `dependencyIds = m.Dependencies.Select(d => d.DependsOnModuleId)`.
4. `isUnlocked = dependencyIds.All(dId => completedModuleIds.Contains(dId))` — every prerequisite must be completed.
5. In `GetModuleContentAsync`, if `!isUnlocked` an `UnauthorizedAccessException("Complete all prerequisite modules first.")` is thrown, which the controller maps to HTTP 403.

DAG Cycle Prevention

`Algorithms/Graph/DagValidator.WouldCreateCycle(adjacency, from, to)` is called in `AddDependencyAsync` before a new edge (module X depends on module Y) is saved.

- `adjacency` is built group-by from existing dependencies: `moduleId → [dependsOnModuleId, ...]`.
- The check asks: starting at `to` (the prerequisite), can we reach `from` (the dependent) by following existing edges? If yes, adding the edge would create a cycle.

- It runs a DFS (`Dfs` method) with a `HashSet<int> visited` guard; returns `true` as soon as `current == target`.
- If a cycle would be created, `ArgumentException("Adding this dependency would create a cycle.")` is thrown and the edge is not saved.

Topological Sort

`DagValidator.TopologicalSort(adjacency)` provides a topological ordering of the module graph (DFS-based, pushing nodes after their neighbors; throws `InvalidOperationException` if a cycle is found). This is available for ordering/graph consumption.

Reordering

`ReorderModuleAsync(pathId, moduleId, moveUp, userId)`:

- Loads all modules of the path ordered by `Order`.
- Finds the target index; computes `swapIdx = moveUp ? idx - 1 : idx + 1`.
- If out of bounds, throws `InvalidOperationException("Module is already at the top/bottom.")`.
- Swaps the `Order` values of the two modules and saves.

Path Deletion (transactional, FK-safe)

`DeleteAsync` runs inside an explicit transaction wrapped in the EF Core execution strategy (needed because SQL Server has `EnableRetryOnFailure()`):

1. Blocks deletion if the path has classrooms or issued certificates.
2. Deletes dependent rows in FK-safe order: student answers → quiz attempts → module dependencies (both directions) → module quiz assignments → progress rows → module objectives/resources/tags → modules → path.
3. Audit logging is best-effort (wrapped in try/catch so a log failure does not fail the delete).

Publish Validation

`PublishModuleAsync` requires:

- Module not archived.
- Title and description non-empty.
- At least one content item (`ContentBody` / `ContentUrl` / `NotesHtml` / any resource).

Module Content Retrieval

`GetModuleContentAsync` returns `ContentUrl`, `ContentType`, `ContentBody`, `NotesHtml`, `PdfUrl`, `resources`, `objectives`, and `tags`, plus `PreviousModuleId` / `NextModuleId` (computed from the module's `Order` position).

Ownership Enforcement

`GetOwnedPathAsync(id, userId)` loads the path and throws `UnauthorizedAccessException` if `path.CreatedById != userId`. Every mutation (update, delete, add/edit/publish/archive/reorder module, add/remove dependency) first calls this helper. Note the `[Authorize(Roles = "Admin,Instructor")]` attribute only checks

the role — according to business rules `Instructor` is forced through the same ownership check, so an admin acting on a path they do not own is also blocked by the same check.

Frontend Graph Visualization

- `frontend/src/utils/graphUtils.ts` builds an adjacency map from dependencies, computes **levels** via a Kahn-style BFS (nodes with in-degree 0 get level 0; each child level = $\max(\text{parent level}) + 1$, queue-driven), and derives edges (`from = dependsOnModuleId`, `to = moduleId`).
- `frontend/src/components/learningPath/PathGraph/PathGraph.tsx` renders the graph as an SVG with nodes positioned per level (`NODE_W=160`, `NODE_H=70`, `COL_GAP=200`, `ROW_GAP=100`).
- `getUnlockedModules` mirrors the backend rule: a module is unlocked when every dependency is in the completed set.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/learningPaths/LearningPathsPage.tsx</code>	Browse/search public paths
Page	<code>frontend/src/pages/learningPaths/LearningPathDetailPage.tsx</code>	Path detail: modules list + graph tab + progress %
Page	<code>frontend/src/pages/learningPaths/LessonPage.tsx</code>	Module content player with unlock gating and inline quiz
Component	<code>frontend/src/components/learningPath/PathCard.tsx</code>	Path listing card
Component	<code>frontend/src/components/learningPath/CreatePathModal.tsx</code>	Create path modal
Component	<code>frontend/src/components/learningPath/PathGraph/PathGraph.tsx</code>	SVG dependency graph
Utils	<code>frontend/src/utils/graphUtils.ts</code>	Level assignment, edges, unlock computation
Service	<code>frontend/src/services/pathService.ts</code>	Path/module/dependency API calls
State	<code>frontend/src/redux/slices/pathSlice.ts</code>	Path + module Redux state

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/LearningPathController.cs	Paths, modules, dependencies, search, reorder endpoints
Service	backend/Services/LearningPath/LearningPathService.cs	Business rules, unlock logic, ownership, deletion transaction
Algorithm	backend/Algorithms/Graph/DagValidator.cs	Cycle detection + topological sort
DTO	backend/DTOs/LearningPath/*	Request/response DTOs for paths, modules, dependencies
Validator	backend/Validators/LearningPath/*	Create path/module validators
Mapping	backend/Mappings/ModuleMappingProfile.cs	Entity ↔ DTO mapping
Interface	backend/Interfaces/Services/ILearningPathService.cs	Service contract

8. Database Implementation

Entity/Table	Purpose	Important Relationship
LearningPaths	Path metadata (title, description, thumbnail, published, public)	FK CreatedById → Users; 1→N Modules, 1→N Classrooms
Modules	Module content + lifecycle flags (draft/published/archived, order, status, difficulty)	FK LearningPathId → LearningPaths; 1→N Progresses, Resources, Objectives, Tags; 1→1 ModuleQuiz
ModuleDependencies	Directed edge: module depends on another module	Composite PK (ModuleId, DependsOnModuleId); both FK → Modules
ModuleResources	Resource links/items per module	FK ModuleId → Modules; ordered by OrderIndex
ModuleObjectives	Learning objectives per module	FK ModuleId → Modules; ordered by OrderIndex
ModuleTags	Tags per module	FK ModuleId → Modules
Progresses	Per-user module completion	FK ModuleId → Modules, UserId → Users

- `Module` has a self-referencing pair of collections (`Dependencies`, `Dependents`) modeled through `ModuleDependency` .
 - `ModuleStatus` enum tracks lifecycle (`NotStarted` → ... → `Completed` → `Archived`).
 - Deleting a path is done in an explicit transaction with FK-safe removal order.
-

9. Security & Authorization

- `GET /api/v1/paths/public` is `[AllowAnonymous]` .
 - Path/module/dependency mutations require `[Authorize(Roles = "Admin,Instructor")]`; deletion requires `Admin` .
 - Ownership re-check in `GetOwnedPathAsync`: instructors may only modify paths where `CreatedById == userId` .
 - Module content access (student) is gated by prerequisite completion — a 403 is returned if the module is locked.
 - Only `IsPublished` modules are returned to students (`includeUnpublished` query controls author visibility of drafts).
-

10. Important Business Rules

1. **Acyclic graph:** Adding a dependency that would create a cycle is rejected.
 2. **Duplicate dependency:** Re-adding the same edge throws `"Dependency already exists."`
 3. **Delete blocked:** A module that other modules depend on cannot be deleted (`DeleteModuleAsync` lists dependents).
 4. **Path delete blocked:** A path with classrooms or certificates cannot be deleted.
 5. **Publish requirements:** Module needs a title, description, and at least one content item; archived modules cannot be (un)published.
 6. **Module title uniqueness:** Duplicate module titles within one path are rejected.
 7. **Reorder bounds:** Reorder fails at top/bottom position.
 8. **Locked content:** Students get 403 on modules whose prerequisites are not completed.
 9. **New paths:** `IsPublic = true` also sets `IsPublished = true` at creation.
-

11. Example of Internal Execution

Step 1: Instructor adds a dependency (prerequisite edge)

- **User Action:** Instructor marks "Module 5 must be completed before Module 7".
- **Frontend:** `POST /api/v1/paths/3/dependencies` with `{ moduleId: 7, dependsOnModuleId: 5 }` .
- **Controller → Service:** `LearningPathService.AddDependencyAsync` .
- **Cycle check:** `DagValidator.WouldCreateCycle(adjacency, 7, 5)` — DFS from node 5 looking for node 7. If a path `5 → ... → 7` exists, the edge would create a cycle and it is rejected.

- **DB:** Otherwise a `ModuleDependencies` row (7, 5) is inserted and a `DEPENDENCY_ADDED` audit entry is logged.

Step 2: Student opens a locked module

- **User Action:** Student clicks "Module 7" in the path detail page (Module 5 not completed yet).
- **Frontend:** `GET /api/v1/paths/3/modules/7`.
- **Service:** `GetModuleContentAsync` loads path with dependencies + student progress; `isUnlocked = dependencyIds.All(completed) → false` because module 5 is not completed.
- **Response:** `UnauthorizedAccessException` → HTTP 403 with message "Complete all prerequisite modules first."
- **Frontend UI:** The request fails and the lesson page is not shown.

Step 3: After completing Module 5, the student opens Module 7 again

- The same call now computes `isUnlocked = true`; the module content, resources, objectives, tags, and previous/next ids are returned and the lesson renders.

12. Files Involved

Frontend

- `frontend/src/pages/learningPaths/LearningPathsPage.tsx`
- `frontend/src/pages/learningPaths/LearningPathDetailPage.tsx`
- `frontend/src/pages/learningPaths/LessonPage.tsx`
- `frontend/src/components/learningPath/PathCard.tsx`
- `frontend/src/components/learningPath/CreatePathModal.tsx`
- `frontend/src/components/learningPath/PathGraph/PathGraph.tsx`
- `frontend/src/utils/graphUtils.ts`
- `frontend/src/services/pathService.ts`
- `frontend/src/services/progressService.ts`
- `frontend/src/redux/slices/pathSlice.ts`

Backend

- `backend/Controllers/LearningPathController.cs`
- `backend/Services/LearningPath/LearningPathService.cs`
- `backend/Algorithms/Graph/DagValidator.cs`
- `backend/DTOs/LearningPath/*`
- `backend/Validators/LearningPath/*`
- `backend/Mappings/ModuleMappingProfile.cs`
- `backend/Interfaces/Services/ILearningPathService.cs`

Database

- `backend/Entities/LearningPath.cs`

- `backend/Entities/Module.cs`
 - `backend/Entities/ModuleDependency.cs`
 - `backend/Entities/ModuleResource.cs`
 - `backend/Entities/ModuleObjective.cs`
 - `backend/Entities/ModuleTag.cs`
 - `backend/Entities/ModuleStatus.cs`
 - `backend/Entities/ModuleDifficulty.cs`
 - `backend/Configurations/*` (module-related EF configurations)
-

13. Functionality

- Create / update / delete learning paths
- Browse public published paths
- Add / edit / delete modules with content, resources, objectives, tags, difficulty, duration
- Publish / unpublish / archive / unarchive modules
- Reorder modules (move up / down)
- Add / remove module dependency edges (prerequisites)
- DAG cycle prevention and topological sort
- Prerequisite-based module unlocking
- Per-user module completed state and path progress percentage
- Path dependency-graph visualization
- Search/filter modules by title, content type, difficulty, draft, archived status